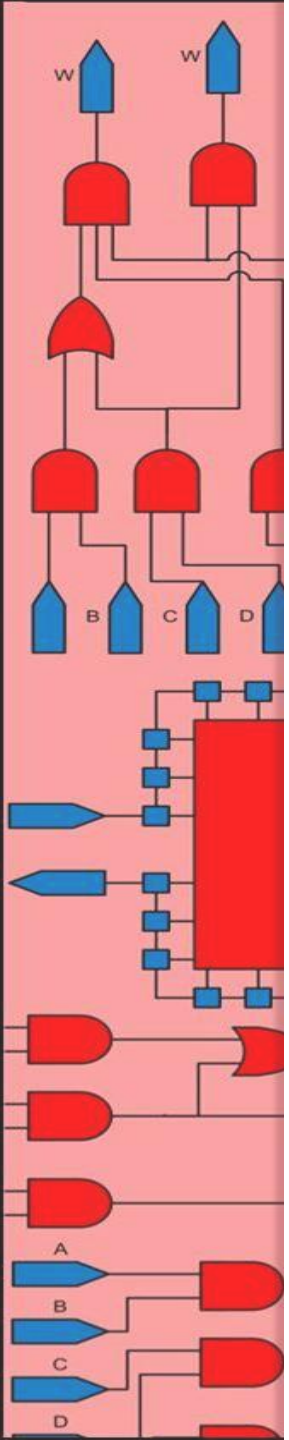


Topic 4

Sequential Components SystemVerilog Constructs



SystemVerilog Constructs

- Concurrent Constructs in Hardware Modules

Primitive/ Module
Instantiations

Assign Statement

The order doesn't matter

`always_ff`

`always_comb`

To model combination components

To model sequential components

SystemVerilog Constructs

- *always_ff*

To describe sequential logic

Executed whenever an **event** in its **sensitivity list** occurs

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        Qout <= 1'b0;
    else
        Qout <= Din;
end
```

Asynchronous Reset

Non-blocking assignment

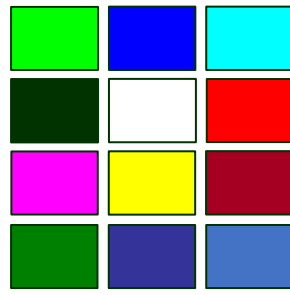
SystemVerilog Constructs

- Blocking & Non-blocking Assignments
 - = (blocking assignment): happens immediately
 - <= (non-blocking assignment): means schedule for later update

```
always_comb begin  
    out_val = in_val;  
end
```

```
always_ff @(posedge clk) begin  
    out_val <= in_val;  
end
```

SystemVerilog Constructs



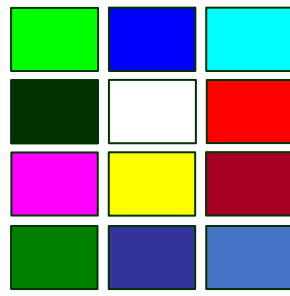
- Blocking & Non-blocking Assignments
 - = (blocking assignment): happens immediately, one after another (like normal programming)
 - <= (non-blocking assignment): happens at the same time at the end of the clock cycle (used in sequential logic)

```
always_ff @(posedge clk) begin
    a = b;    // blocking
    c = a;
end
```

- First a gets b
- Then c gets the new value of a

```
always_ff @(posedge clk) begin
    a <= b;    // non-blocking
    c <= a;
end
```

- Both updates are scheduled together
- a gets b, but at the same time c still gets the old value of a (before the clock edge)



SystemVerilog Constructs

- Packed and Unpacked Arrays
 - Packed array ([msb:lsb])
 - Represents a bit vector (contiguous bits)
 - Hardware: like a bus
 - Can do bit slicing, concatenation, arithmetic, etc.
 - Unpacked array ([size])
 - Represents a collection of separate elements
 - Hardware: like a memory or register file
 - Index one element at a time

```
logic [DATA_WIDTH-1:0] regs [NUM_REGS];
```

Packed

Unpacked

`logic [15:0][7:0] mem;` → one register (128 bits), split into chunks

`logic [7:0] mem [15:0];` → 16 registers (8 bits each), a true memory/register file

SystemVerilog Constructs

- Integer Data Types

shortint	2-state SystemVerilog data type, 16 bit signed integer
int	2-state SystemVerilog data type, 32 bit signed integer
longint	2-state SystemVerilog data type, 64 bit signed integer
byte	2-state SystemVerilog data type, 8 bit signed integer or ASCII character
bit	2-state SystemVerilog data type, user-defined vector size
logic	4-state SystemVerilog data type, user-defined vector size
reg	4-state Verilog-2001 data type, user-defined vector size
integer	4-state Verilog-2001 data type, 32 bit signed integer
time	4-state Verilog-2001 data type, 64-bit unsigned integer

IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Standard 1800-2017, Dec. 2017.

SystemVerilog LRM (Language Reference Manual)

SystemVerilog Constructs

- Type Definition (*typedef*)

Makes code more readable, reusable, and easier to maintain

an *enum* is a data type that lets you define a set of named values instead of just using numbers

Enums

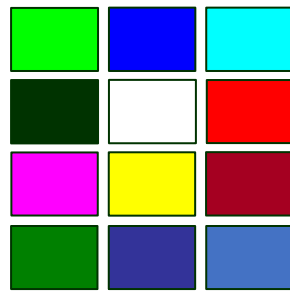
```
typedef enum logic [1:0] {  
    IDLE,  
    READ,  
    WRITE,  
    DONE  
} state_t;  
  
state_t present_state, next_state;
```

Structs

```
typedef struct packed {  
    logic [7:0] opcode;  
    logic [3:0] addr;  
    logic      valid;  
} instr_t;  
  
instr_t instr1, instr2;
```

Alias for a type

```
typedef logic [31:0] word_t;  
  
word_t a, b, c;
```

Testbench

Non-synthesizable SystemVerilog Constructs

- *initial* statement
- Built-in functions
 - Input Data generation
 - Deterministic data
 - Random data: \$urandom, \$urandom_range(min, max), ...
 - Read data from a file
 - Output Analysis
 - \$display, \$monitor
 - Write the results to a file
 - Waveform
 - To pause or terminate simulation: \$stop, \$finish
 - To return the current simulation time: \$time
 - To wait for a specific time: #delay

A concurrent process

Executed only once at simulation time 0

\$fopen, \$fdisplay, \$fscanf, \$feof, and \$fclose

Testbench

- Reading from a file & Writing to a file

```
module tb_file_io_rw;
```

```
int in_fd, out_fd;
```

file handles

```
int status;
```

fscanf return status

```
int fdata_in;
```

variable for reading values

```
...
```

```
initial begin
```

```
in_fd = $fopen("input.txt", "r");
```

```
if (in_fd == 0) begin
```

```
    $display("ERROR: Could not open input.txt for reading.");
```

```
    $finish;
```

```
end
```

```
out_fd = $fopen("results.txt", "w");
```

```
if (out_fd == 0) begin
```

```
    $display("ERROR: Could not open results.txt for writing.");
```

```
    $finish;
```

```
end
```

```
while (!$feof(in_fd)) begin
```

```
    status = $fscanf(in_fd, "%b\n", fdata_in);
```

```
    if (status == 1) begin
```

Reading from a file

```
        in_a = fdata_in [ 7:0];
```

```
        in_b = fdata_in [15:8];
```

```
        @(posedge clk);
```

```
        $fdisplay(out_fd, "Output=%0b", out_w);
```

Writing to a file

```
    end
```

```
end
```

```
$fclose(in_fd);
```

```
$fclose(out_fd);
```

```
$finish;
```

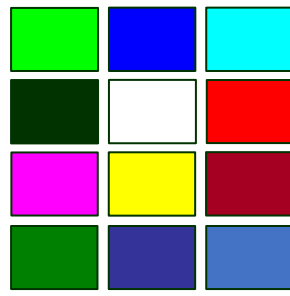
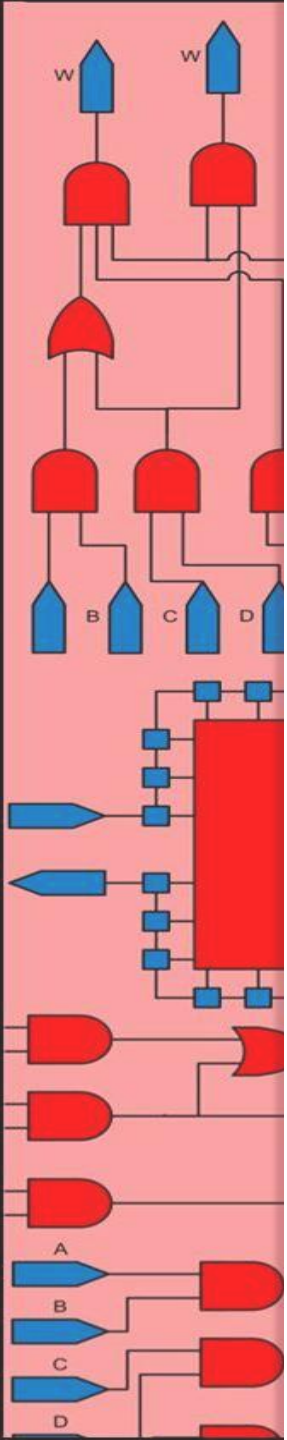
```
end
```

```
endmodule
```

Testbench

- Reading from a file & Writing to a file

Task	Description
\$fopen	Opens a file and returns a non-zero integer if the file opens successfully, 0 otherwise
\$feof	Checks if the end of a file has been reached. Returns 1 if EOF, 0 otherwise
\$fscanf	Reads data from a file and returns the number of successfully read items, or -1 at EOF
\$fdisplay	Writes data to a file and appends a newline at the end
\$fclose	Closes an open file and releases simulator resources



Conclusions

- Concurrent constructs in hardware modules
 - *always_ff*
- Blocking & non-blocking assignments
- Packed and unpacked arrays
- Integer data types
- Type definition
- Testbench constructs
 - Reading from a file & Writing to a file